

Final Report — CAGS Project ARES

(Autonomous Repair Exploration System)

◆ Team Members

- **Juan Camilo Gómez Duarte** — Project Leader, responsible for the design, training, and direction of the AI backend.
 - **Gustavo Adolfo Castillo Páez** — Responsible for the design, assembly, and technical documentation of the CAGS_F413 drone.
 - **Ángel Daniel Scarpetta Ruiz** — Developer of the administrative backend and in charge of system data management and control.
 - **Sara Sofía Alzate Charry** — Frontend developer of the control interface (“NASA Interface”) and main visualization panel.
-

◆ Project Overview

CAGS – Project ARES (Autonomous Repair Exploration System) is an integrated system focused on the autonomous operation of drones specialized in inspection and repair in critical environments such as space stations, remote areas, or hazardous industrial settings. The ecosystem combines artificial intelligence, real-time communication technology, web-based control, and distributed processing to ensure secure, auditable, and scalable operations.

◆ General Objective

Develop a complete platform integrating AI, backend, frontend, and drone systems to autonomously perform diagnostic and repair missions with real-time human monitoring.

◆ System Architecture

The system is composed of **four main layers**:

1. Physical Layer / Drone (Gustavo Castillo)

- Flight hardware and sensors.
- Navigation and telemetry control system.
- Real-time data capture (temperature, energy, position, failures).
- Communication link with the backend via WebSocket and MQTT.

2. Logic Layer / Backend (Ángel Scarpetta)

- Implemented using Django REST Framework (DRF).
- Manages drone data, telemetry, inventory, events, repairs, and audits.
- Authentication and authorization based on JWT and user roles.
- Access control through RolePermission class.

- Database: SQLite (development) / PostgreSQL (production).
- Security using Argon2, HTTPS, secure cookies, and encrypted logs.

3. Interface Layer / Frontend (Sara Alzate)

- Developed with React (Vite) + React-Bootstrap.
- Real-time communication using WebSocket (Django Channels).
- Data visualization in a “NASA Control Center” style dashboard.
- Modules: Dashboard, Alarms, Repairs, Inventory, Logs, Training, and Configuration.

4. Artificial Intelligence Layer (Juan Camilo Gómez Duarte)

- AI backend responsible for interpreting telemetry data and predicting failures.
- Model training for autonomous diagnostics and repair prioritization.
- Bidirectional communication with the backend via REST API and WebSockets.
- Continuous learning capability using datasets uploaded from the frontend.

◆ Roles and Responsibilities

Member	Role	Main Responsibilities
Juan Camilo Gómez Duarte	Project Leader / AI Backend	General coordination, AI architecture, data flow control, integration between backend and prediction models.
Gustavo Adolfo Castillo Páez	Drone Engineer	Drone design, sensor documentation, navigation systems, telemetry, and physical assembly.
Ángel Daniel Scarpetta Ruiz	Backend Developer	Creation of the administrative backend (Django), role management, audits, telemetry, and data security.
Sara Sofía Alzate Charry	Frontend Developer	Development of the control interface, connection to backend, and UX/UI design.

◆ Main Modules

1. Users and Roles Module

- Roles: admin, engineer, operator.
- Permission-based access:
- **Admin:** full system control.
- **Engineer:** technical editing (drones, events, inventory, repairs).

- **Operator:** read-only access.
 - Security with short-lived JWT and refresh tokens.
- 2. **Drone / Telemetry Module**
 - Registers serial number, connection status, battery, temperature, firmware version, and location.
 - Associates damage reports with severity, type, and evidence (photo/video).
 - Real-time updates via WebSocket backend communication.
- 3. **Repairs Module**
 - Detailed record of each repair (date, materials, status, multimedia evidence).
 - Association with prior damage reports.
 - Digital signature and integrity hash for audit purposes.
- 4. **Audit Module**
 - Records critical events and anomalies.
 - Immutable logs with verification hashes.
 - Exportable in JSON or CSV.
- 5. **Inventory Module**
 - Tracks parts, tools, and materials.
 - Filters by category, criticality, and availability.
 - Stock control restricted to engineer roles.
- 6. **Events / Alarms Module**
 - Real-time visualization of critical alarms.
 - Notification system with priority levels.
 - “Acknowledge” option and history tracking.
- 7. **Training and Command Module**
 - Allows sending operation and training commands to drones.
 - Upload of datasets and AI performance metrics.
 - Historical log of training sessions and results.

◆ **Security and Standards**

- Mandatory HTTPS/TLS.
 - MFA for critical roles.
 - CSRF/XSS/Clickjacking protection via CSP and SameSite policies.
 - Encryption at rest (DB encryption / field-level encryption).
 - Digitally signed logs for auditing.
 - Periodic backups and disaster recovery plan.
 - OWASP Top 10 evaluation before deployment.
-

◆ Key Technical Integrations

- MQTT / RabbitMQ — Low-latency communication channel.
 - Swagger / OpenAPI — Interactive API documentation.
 - ELK Stack — Log storage and analysis system (Elasticsearch, Logstash, Kibana).
 - Django Channels + Socket.io — Real-time bidirectional communication.
-

◆ Current Project Status

- ✓ Fully functional backend with secure authentication and structured endpoints.
 - ✓ Frontend in final development phase, connected to backend via WebSocket and REST API.
 - ✓ Drones configured and transmitting telemetry correctly.
 - ✓ AI trained locally for predictive diagnostics and repair execution.
-

◆ General Conclusion

CAGS – Project ARES represents a major advancement in the integration of artificial intelligence, robotics, and distributed computing applied to autonomous exploration and repair.

The system combines technical precision, security, and modularity, making it adaptable to multiple environments (space, industrial, or rescue).

The project consolidates a complete **AI ↔ Backend ↔ Frontend ↔ Drone** flow, ready for operational testing and future expansion.

◆ Bibliography and References

- **NASA (National Aeronautics and Space Administration)**. Documentation and visual inspiration on autonomous operations and mission telemetry.
- **Project Gemini Archives (NASA, 1965–1966)**. Historical reference for automated orbital maintenance design and logic.
- **OpenAI — ChatGPT (2025)**. Assistance with technical documentation structure and report writing.
- **xAI — Grok AI (2025)**. Support in backend algorithm optimization and logical structure improvement.
- **Google Gemini (2025)**. Visual generation support for imagery and simulation materials.